

Các kỹ thuật duyệt List trong Java



NỘI DUNG

1. Vì sao cần nhiều cách duyệt?

2. Duyệt bằng for (cổ điển)

3. Duyệt bằng foreach

4. Duyệt bằng Iterator

5. Duyệt xuôi & ngược

6. So sánh nhanh

7. Lỗi thường gặp

8. Kết luận

Vấn đề đặt ra

Vì sao cần nhiều cách duyệt?

Duyệt để đọc dữ liệu

Duyệt để sửa / xóa phần tử

Duyệt theo thứ tự xuôi / ngược

Không phải lúc nào cũng chỉ cần duyệt để in ra dữ liệu. Có những trường hợp cần sửa, xóa phần tử khi đang duyệt – và mỗi kỹ thuật sẽ phù hợp với từng tình huống khác nhau.

List mẫu

Chuẩn bị dữ liệu ví dụ

```
List<String> list = new ArrayList<>();  
  
list.add("Java");  
  
list.add("Python");  
  
list.add("C++");
```

Trong suốt video này, chúng ta sẽ dùng chung một List ví dụ để so sánh các cách duyệt.

Duyệt bằng for (cổ điển)

for (index-based)

```
for(int i = 0; i < list.size(); i++) {  
  
    System.out.println(list.get(i));  
  
}
```

Đặc điểm:

- Truy cập bằng index
- Dễ kiểm soát vị trí

Cách này quen thuộc nhất với người mới học. Tuy nhiên, với LinkedList thì cách duyệt này kém hiệu quả.

Ưu & nhược điểm for

for thường

Ưu điểm

- Dễ hiểu
- Dùng index

Nhược điểm

- Dài dòng
- Không an toàn khi xóa

Nếu xóa phần tử trong vòng for không cẩn thận, rất dễ gặp lỗi logic.

Duyệt bằng foreach

for-each

```
for(String s : list) {  
  
    System.out.println(s);  
  
}
```

Đặc điểm:

- Gọn, dễ đọc
- Không dùng index

foreach là cách duyệt phổ biến nhất hiện nay khi chỉ cần đọc dữ liệu.

Lưu ý foreach

Không dùng để xóa phần tử

❌ Ví dụ lỗi:

```
for(String s : list) {  
  
    if(s.equals("Java")) {  
  
        list.remove(s); // Lỗi  
  
    }  
  
}
```

Duyệt foreach mà xóa phần tử sẽ gây ra lỗi **ConcurrentModificationException**.

Duyệt bằng Iterator

Iterator

```
Iterator<String> it = list.iterator();

while(it.hasNext()) {

    String s = it.next();

    System.out.println(s);

}
```



Đặc điểm:

- Duyệt an toàn
- Có thể xóa phần tử

Iterator là cách duyệt an toàn khi cần xóa phần tử trong quá trình duyệt.

Xóa phần tử với Iterator

remove()

```
if(s.equals("Java")) {  
  
    it.remove();  
  
}
```

 **Lưu ý:** phải gọi remove thông qua iterator, không được gọi list.remove().

ListIterator là gì?

ListIterator

```
ListIterator<String> it = list.listIterator();
```



Chỉ dùng cho List



Duyệt 2 chiều



Thêm / sửa / xóa

ListIterator mạnh hơn Iterator rất nhiều nhưng cũng phức tạp hơn.

Duyệt xuôi & ngược

next() & previous()

Duyệt xuôi

```
while(it.hasNext()) {  
  
    System.out.println(it.next());  
  
}
```

Duyệt ngược

```
while(it.hasPrevious()) {  
  
    System.out.println(it.previous());  
  
}
```

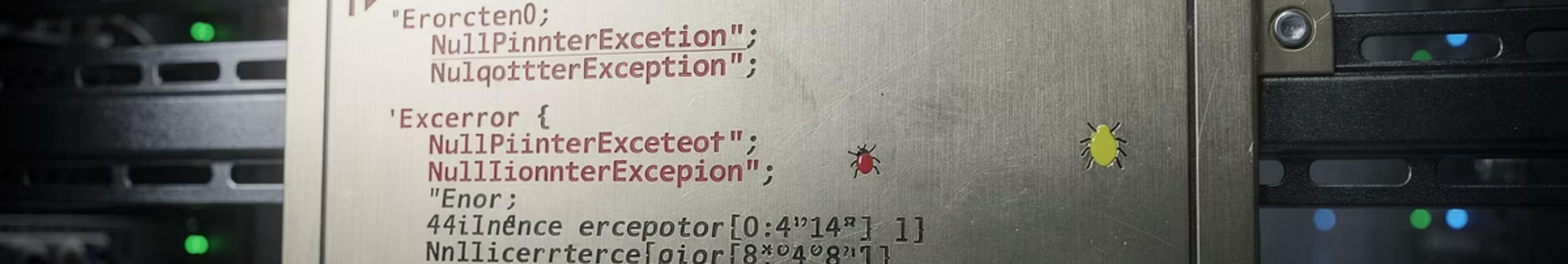
Đây là điểm mạnh nhất của ListIterator – duyệt danh sách theo cả hai chiều.

So sánh nhanh

Nên dùng cách nào?

Kỹ thuật	Khi nào dùng
for	Cần index
foreach	Chỉ đọc
Iterator	Xóa phần tử
ListIterator	Sửa, thêm, duyệt ngược

Chọn đúng kỹ thuật duyệt sẽ giúp code an toàn và hiệu quả hơn.



```
1 // "Error ten 0;  
    NullPinnterExcetion";  
    NulqottterException";  
  
'Excerror {  
    NullPiinterExceteot";  
    NullIionnterExcepcion";  
    "Enor";  
    44iInñnce ercepotor[0:4"14" 1}  
    Nnllicertrterce[pior[8*0408"1]
```

Lỗi thường gặp

Những lỗi cần tránh

- ☐ Xóa phần tử trong foreach
- ☐ Dùng for với LinkedList
- ☐ Nhầm Iterator và ListIterator

Những lỗi này rất thường gặp ở người mới học List.

Tổng kết

Kết luận

01

Có nhiều cách duyệt List

02

Mỗi cách có mục đích riêng

03

Hiểu để dùng đúng